

A Highly-Parallel TSP Solver for a GPU Computing Platform

Noriyuki Fujimoto¹ and Shigeyoshi Tsutsui²

¹ Osaka Prefecture University, 1-1 Gakuen-Cho, Naka-ku, Sakai-shi,
Osaka, 599-8531, Japan

`fujimoto@mi.s.osakafu-u.ac.jp`

² Hannan University, 5-4-33 Amamihigashi, Matsubara,
Osaka, 580-8502, Japan

`tsutsui@hannan-u.ac.jp`

Abstract. The traveling salesman problem (TSP) is probably the most widely studied combinatorial optimization problem and has become a standard testbed for new algorithmic ideas. Recently the use of a GPU (Graphics Processing Unit) to accelerate non-graphics computations has attracted much attention due to its high performance and low cost. This paper presents a novel method to solve TSP with a GPU based on the CUDA architecture. The proposed method highly parallelizes a serial metaheuristic algorithm which is a genetic algorithm with the OX (order crossover) operator and the 2-opt local search. The experiments with an NVIDIA GeForce GTX285 GPU and a single core of 3.0 GHz Intel Core2 Duo E6850 CPU show that our GPU implementation is about up to 24.2 times faster than the corresponding CPU implementation.

Keywords: parallel metaheuristic, genetic algorithm, GPGPU.

1 Introduction

The traveling salesman problem [1,8] (TSP) is probably the most widely studied combinatorial optimization problem and has become a standard testbed for new algorithmic ideas [6]. Recently the use of a GPU (Graphics Processing Unit) to accelerate non-graphics computations has attracted much attention due to its high performance and low cost.

This paper presents a novel method to solve TSP with a GPU based on the CUDA architecture [10]. Especially for CUDA, the proposed method highly parallelizes a serial metaheuristic algorithm which is a genetic algorithm (GA for short) with the OX (order crossover) operator [5] and the 2-opt local search [4].

Genetic algorithms have obvious parallelism among individuals. However, the parallelism is not enough to obtain high performance of a GPU. To utilize an advantage of "many-thread" architecture of CUDA, we extract not only the parallelism among individuals but also another parallelism in the processing of each individual. That is, we parallelize the execution of each OX operator and each 2-opt local search, too.

To evaluate the effectiveness of the proposed method, we conduct some experiments for TSPLIB [13] benchmark problem instances using an NVIDIA GeForce GTX285 GPU and a 3.0 GHz Intel Core 2 Duo E6850 CPU. The experimental results show that our GPU implementation is about up to 24.2 times faster than the corresponding CPU implementation.

Quite recently, CUDA has been successfully used to accelerate various applications in scientific fields such as fluid dynamics, image processing, and simulations. However, in the field of genetic algorithms to solve TSP, almost no result is known except for Sanci's result [14]. This is because genetic algorithms have special property of frequent random access to large data-structures, which is not the case in the other successful fields. Sanci achieved at most 4.9 times speedup using an NVIDIA GeForce 9800M GPU and a 2.13GHz Intel Core2 Duo P8400 CPU.

The remainder of this paper is organized as follows. Section 2 presents the proposed algorithm. Experiments to show the performance of the proposed algorithm are reported in Section 3. Section 4 gives some concluding remarks and future works. Due to the limited space, this paper include no description on CUDA. Readers unfamiliar with CUDA are recommended the literature [10,12].

2 The Proposed Algorithm

2.1 An Overview of the Proposed Algorithm

Listing 1.1 shows a pseudo code of a serial GA program to solve a TSP instance with the OX operator and 2-opt local search. Usually, a crossover operator generates two offspring from two parents. However, in the proposed method, we generate only one child from two parents. In Step 18 of Listing 1.1, the comparison of costs is performed like a tournament selection with size 2. However, each comparison is performed between individuals $s1[i]$ and $s2[i]$ which have the same index i . Please remember here that $s2[i]$ is generated from $s1[i]$ as one of its parents (the other parent $s1[j]$ was chosen randomly). Since a parent and a child have partly similar substrings, this comparison scheme can be expected to maintain population diversity like the deterministic crowding proposed by Mahfoud [11]. Using one child from two parents was already proposed for designing the well known GENITOR algorithm by Whitley et al. [16].

Genetic algorithms have obvious parallelism among individuals. So, also in the case of the serial GA program, **for** loops in line 5, 11, and 16 can be executed in parallel. Before parallelizing the serial GA program, we conducted some preliminary experiments to investigate how many individuals are required to solve a TSP instance efficiently. The results show that at most tens of individuals are enough for instances at most 500 cities from TSPLIB benchmark. On the CUDA architecture, the only way to hide memory access latency is to execute other threads when some threads are stalled due to memory access latency [12]. At a time, a CUDA GPU can make 30720 threads of all the running threads active. So, to hide memory access latency efficiently, at least hundred thousands of threads should be created. Therefore, the parallelism among individuals is not sufficient at all to extract high performance of a GPU.

Listing 1.1. A serial GA with OX operator and 2-opt local search for TSP

```

1 // s1[0 .. n - 1] and p1[0 .. n - 1] :
2 // p1[i] is the length of tour s1[i]
3 // s2[0 .. n - 1] : buffer for individuals generated by
4 // OX and 2-opt
5 // p2[0 .. n - 1] : p2[i] is the length of tour s2[i]
6 generation = 0;
7 for (i = 0; i < populationSize; i++) {
8     s1[i] = a random tour;
9     p1[i] = evaluate(s1[i]);
10    if (p1[i] is acceptable) return i; // found solution is s1[i]
11 }
12 while (generation < maxGeneration) {
13     for (i = 0; i < populationSize; i++) {
14         j = a random integer such that
15         0 <= j < populationSize and j != i;
16         s2[i] = OX(s1[i], s1[j]);
17         2OPT_best(s2[i]);
18     }
19     for (i = 0; i < populationSize; i++) {
20         p2[i] = evaluate(s2[i]);
21         if (p2[i] < p1[i]) {
22             s1[i] = s2[i];
23             p1[i] = p2[i];
24         }
25         if (p1[i] is acceptable) return i; // found solution is s1[i]
26     }
27     generation++;
28 }
29 return -1; // solution is not found

```

Hence, the proposed method parallelizes the serial GA program furthermore. That is, we also parallelized the OX operator and 2-opt local search. To realize this approach, the proposed method adopted the following structure:

- The proposed method runs m thread blocks concurrently where m is the number of individuals.
- Each individual is processed by a thread block with n threads where n is the number of cities of a given TSP instance.
- Each thread block concurrently performs the OX operator and 2-opt local search in the best improvement manner [6] for its individual.

Listing 1.2 shows a high level description of the proposed method.

The 2-opt local search in the best improvement manner can be easily parallelized. So, the next subsection describes how the proposed method parallelizes the OX operator.

2.2 Parallelization of the OX Operator

OX constructs an offspring by choosing a subsequence of one parent and preserving the relative order of cities of the other parent. Listing 1.3 shows an implementation of OX in the C programming language. The time complexity of OX is $O(n^2)$ where n is the number of cities. It is not so clear whether OX has

Listing 1.2. A high level description of the proposed method

```

1  --global-- void kernel(int *s, int *d, int *ps, int *pd,
2                      int *found)
3  {
4      int i = blockIdx.x;
5
6      if (threadIdx.x == 0) {
7          j = a random integer such that
8              0 <= j < populationSize and j != i;
9      }
10     __syncthreads();
11
12     // s_si, s_sj, s_pi, s_pj, and s_d are pointers to shared mem.
13     copy s[i] to *s_si in parallel within this block;
14     copy s[j] to *s_sj in parallel within this block;
15     if (threadIdx.x == 0) copy ps[i] to *s_pi;
16     *s_d = OX(s_si, s_sj) in parallel within this block;
17     2OPT_best(*s_d) in parallel within this block;
18     p = evaluate(s_d) in parallel within this block;
19
20     __syncthreads();
21     if (p < *s_pi) {
22         d[i] = *s_d in parallel within this block;
23         if (threadIdx.x == 0) pd[i] = p;
24     } else {
25         d[i] = *s_si in parallel within this block;
26         if (threadIdx.x == 0) p = pd[i] = ps[i];
27     }
28     if (threadIdx.x == 0) {
29         if (p is acceptable) *found = i; // found solution is d[i]
30     }
31 }
32
33 int main()
34 {
35     // s1, s2, p1, and p2 are pointers to main memory.
36     // d_s1, d_s2, d_p1, d_p2, and d_found are pointers to VRAM.
37     generation = 0;
38     for (i = 0; i < populationSize; i++) {
39         s1[i] = a random tour; p1[i] = evaluate(s1[i]);
40         if (p1[i] is acceptable) return i; // found solution is s1[i]
41     }
42     copy s1[*] to d_s1[*]; copy p1[*] to d_p1[*];
43     while (generation < maxGeneration) {
44         set *d_found to -1;
45         int grid = populationSize; int block = the number of cities;
46         kernel<<< grid, block >>>(d_s1, d_s2, d_p1, d_p2, d_found);
47         if (*d_found >= 0) { acceptable solution is found; break; }
48         swap(d_s1, d_s2); // swap pointers only (double buffering)
49         swap(d_p1, d_p2); // swap pointers only (double buffering)
50         generation++;
51     }
52 }

```

parallelism. So, we propose a parallelized OX. Our parallelized OX generates the same computational results as the original OX. Listing 1.4 shows our parallelization of OX in the C programming language with 'parallel for' construct to indicate parallelism. The total computational amount of our parallelizable OX is the same as the serial time complexity of ordinary OX. Our parallelization uses a prefix sums operation (or scan operation) [2,3] which is a well-known

Listing 1.3. The OX operator

```

1 void OX(int n, int *s1, int *s2, int *d)
2 // s1[0 .. n - 1] and s2[0 .. n - 1] : given two individuals
3 // d[0 .. n - 1] : buffer for a generated individual
4 {
5     Set cut1 and cut2 randomly s. t.
6     0 <= cut1, cut2 < n and cut1 != cut2;
7     for(int j = 0; j < n; j++) d[j] = s1[j];
8     for(int j = cut1; j < cut2; j++){
9         for(int pos = 0; pos < n; pos++) if (s2[j] == d[pos]) break;
10        int next = pos + 1;
11        while(next != cut2){
12            if (pos == n) pos = 0;
13            if (next == n) next = 0;
14            d[pos++] = d[next++];
15        }
16    }
17    for(int j = cut1; j < cut2; j++) d[j] = s2[j];
18 }

```

Listing 1.4. OX in a parallelizable fashion

```

1 void PrefixSums(int n, int *s, int *d, int begin = 0)
2 {
3     d[begin] = s[begin];
4     for (int i = begin + 1; i < n; i++) d[i] = d[i - 1] + s[i];
5     if (begin > 0) {
6         d[0] = d[n - 1] + s[0];
7         for (int i = 1; i < begin; i++) d[i] = d[i - 1] + s[i];
8     }
9 }
10
11 void ParallelizableOX(int n, int *s1, int *s2, int *d)
12 // s1[0 .. n - 1] and s2[0 .. n - 1] : given two individuals
13 // d[0 .. n - 1] : buffer for a generated individual
14 {
15     Set cut1 and cut2 randomly s. t.
16     0 <= cut1, cut2 < n and cut1 != cut2;
17     for (int j = 0; j < n; j++) do in parallel {
18         for (int pos = cut1; pos < cut2; pos++)
19             if (s1[j] == s2[pos]) break;
20         ps[j] = (pos >= cut2) ? 1 : 0;
21     }
22     PrefixSums(n, ps, to, (cut2 < n) ? cut2 : 0);
23     for (int j = 0; j < n; j++) do in parallel {
24         to[j]--;
25         if (to[j] < n - cut2) to[j] += cut2;
26         else to[j] -= (n - cut2);
27     }
28     for (int j = 0; j < n; j++) do in parallel
29         if (ps[j]) d[to[j]] = s1[j];
30     for (int j = cut1; j < cut2; j++) do in parallel d[j] = s2[j];
31 }

```

Table 1. Comparison between the proposed CUDA program and the corresponding CPU program in case that population size is 60, the maximum number of generations is 1000, and acceptable error ratio to the optimal solution is 0.5%

problem instance	CPU			GPU			speedup ratio
	#success in 10 trials	avg. exec. time(sec)	avg. err. (%)	#success in 10 trials	avg. exec. time(sec)	avg. err. (%)	
gr120	10	5.7578	0.44	10	0.5156	0.41	11.2
pr124	10	3.3625	0.23	10	0.2297	0.30	14.6
bier127	10	5.6109	0.37	10	0.4563	0.35	12.3
ch130	10	6.8593	0.28	10	0.5110	0.35	13.4
pr136	10	6.5781	0.44	10	0.4860	0.28	13.5
gr137	10	5.9515	0.27	10	0.4813	0.41	12.4
pr144	10	5.1062	0.21	10	0.2954	0.24	17.3
ch150	10	11.5516	0.37	10	0.7594	0.37	15.2
pr152	10	6.9219	0.40	10	0.4579	0.41	15.1
u159	10	9.8500	0.26	10	0.5656	0.22	17.4
brg180	10	26.7625	0.00	10	1.7281	0.00	15.5
d198	10	21.7875	0.39	10	1.4109	0.43	15.4
kroA200	10	29.6437	0.38	10	1.4891	0.40	19.9
kroB200	10	31.4468	0.38	10	1.5828	0.38	19.9
gr202	10	32.0156	0.43	10	1.9438	0.40	16.5
ts225	10	30.4671	0.38	10	1.2578	0.38	24.2
tsp225	10	45.8062	0.33	10	2.5688	0.41	17.8
pr226	10	27.1046	0.35	10	1.3860	0.39	19.6
gr229	10	49.4391	0.41	10	3.0172	0.42	16.4
gil262	10	60.4265	0.38	10	3.9079	0.41	15.5
pr264	10	34.5985	0.35	10	2.6063	0.40	13.3
a280	10	54.8969	0.35	10	4.0703	0.39	13.5
pr299	10	65.8516	0.45	10	4.7344	0.44	13.9
lin318	10	91.0219	0.41	10	6.8016	0.43	13.4
rd400	10	304.5594	0.46	10	22.8266	0.45	13.3
fl417	10	150.0360	0.47	10	11.6625	0.48	12.9
gr431	9	332.1944	0.45	9	29.9359	0.48	11.1
pr439	10	201.9344	0.45	10	17.2250	0.45	11.7
pcb442	9	330.1320	0.47	8	35.4219	0.46	9.3
d493	10	461.7297	0.46	9	48.5052	0.48	9.5

data-parallel primitive with wide variety of applications [2] and can be efficiently executed in parallel [7]. Also for CUDA GPUs, an efficient parallel implementation of prefix sums is provided by Sengupta et al. [15]. As for prefix sums operations, our implementation uses Sengupta et al.'s CUDA implementation.

3 Experiments

This section compares the performance of the proposed CUDA program with a CPU program that performs the same computation. For each test, a single core of 3.0 GHz Intel Core2 Duo E6850 and NVIDIA GeForce GTX285 was used. The

OS used is Windows XP Professional SP3 with NVIDIA graphics driver Version 195.62. For compilation, Microsoft Visual Studio 2008 Professional Edition with optimization option /O2 and CUDA 2.3 SDK were used.

Table 1 shows the performance of the proposed GPU algorithm and the corresponding CPU algorithm for problem instances from TSPLIB benchmark with at least 120 cities and at most 512 cities. TSPLIB provides the optimal lengths of tours for these problem instances. We measured the execution time for our GPU (CPU) program to find an acceptable solution. An acceptable solution is a solution with tour length within a factor of $(1 + \text{a given acceptable error ratio})$ to the optimal tour length. In our experiments, acceptable error ratio is 0.5%. For each problem instance, the measurement was conducted 10 times consecutively and the average value of successful trials of the 10 trials was adopted where successful trial is a trial such that an acceptable solution is found within 1000 generations of our GA. The speedup ratio is the ratio of the execution time of CPU to that of GPU. The speedup ratio indicates how the GPU program is faster than the CPU program.

We can see the proposed algorithm achieves 24.2 times speedup at the maximum compared with a single core of the CPU. We can also see that the performance of the CUDA program is maximized in case that the number of cities is from 200 to 226. This is because the number of active threads is roughly maximized for such number of cities. This can be verified by the fact that the implementation of the proposed method uses 25 registers per thread and $(20n + 28)$ bytes shared memory per thread block where n is the number of cities.

4 Conclusion and Future Work

A new parallel implementation has been proposed for solving the traveling salesman problem on the NVIDIA CUDA GPU architecture. The proposed algorithm is a highly parallel variant of a genetic algorithm with the OX operator and 2-opt local search. The proposed GPU algorithm has achieved 24.2 times speed up to the corresponding CPU algorithm.

Our experiments was conducted for problem instances at most 512 cities. This is mainly because 2-opt heuristic is insufficient for larger problem instances. Therefore, one of future works is to develop a CUDA implementation for much larger problem instances by parallelizing more sophisticated heuristics (e.g., Lin-Kernighan heuristic [9]) for CUDA.

References

1. Applegate, D.L., Bixby, R.E., Chvátal, V., Cook, W.J.: The Traveling Salesman Problem: A Computational Study. Princeton University Press, Princeton (2007)
2. Blleloch, G.E.: Scans as Primitive Parallel Operations. *IEEE Transactions on Computing* 38(11), 1526–1538 (1989)
3. Blleloch, G.E.: Vector Models for Data-Parallel Computing. MIT Press, Cambridge (1990)

4. Croes, G.A.: A Method for Solving Traveling Salesman Problems. *Operations Research* 6, 791–812 (1958)
5. Davis, L.: Applying Adaptive Algorithms to Epistatic Domains. In: *Proc. of the International Joint Conference on Artificial Intelligence*, pp. 162–164 (1985)
6. Hoos, H.H., Stützle, T.: *Stochastic Local Search: Foundations and Applications*. Elsevier, Amsterdam (2005)
7. JaJa, J.: *An Introduction to Parallel Algorithms*. Addison-Wesley Professional, Reading (1992)
8. Lawler, E.L., Lenstra, J.K., Rinnooy Kan, A.H.G., Shmoys, D.B.: *The Traveling Salesman Problem: A Guided Tour of Combinatorial Optimization*. Wiley, Chichester (1985)
9. Lin, S., Kernighan, B.W.: An Effective Heuristic Algorithm for the Traveling-Salesman Problem. *Operations Research* 21, 498–516 (1973)
10. Lindholm, E., Nickolls, J., Oberman, S., Montrym, J.: NVIDIA Tesla: A Unified Graphics and Computing Architecture. *IEEE Micro* 28(2), 39–55 (2008)
11. Mahfoud, S.: A Comparison of Parallel and Sequential Niching Methods. In: *Proc. of the International Conference on Genetic Algorithms*, pp. 136–143 (1995)
12. NVIDIA: *CUDA Programming Guide 3.1* (2010), http://www.nvidia.com/object/cuda_develop.html
13. Reinelt, G.: TSPLIB: A Traveling Salesman Problem Library. *ORSA Journal on Computing* 3, 376–384 (1991)
14. Sanci, S.: *A Parallel Algorithm for Flight Route Planning on GPU Using CUDA*, Master Thesis, Middle East Technical University (April 2010)
15. Sengupta, S., Harris, M., Garland, M.: Efficient Parallel Scan Algorithms for GPUs. NVIDIA Technical Report NVR-2008-003 (2008)
16. Whitley, L.D., Starkweather, T., Fuquay, D.: Scheduling Problems and Traveling Salesman Problem: The Genetic Edge Recombination Operator. In: *Proc. of the International Conference on Genetic Algorithms*, pp. 133–140 (1989)